

시스템 정의서 & 배포 명세서

바버샵 AI 마케팅 자동화 SaaS · 백엔드 기술 문서

아키텍처 · AI 에이전트 파이프라인 · 데이터 · Azure 리소스 · 배포 · 운영 · 유지보수

문서 버전 v1.0	기준일 2026-07-14	대상 코드 bybaek-backend	브랜치 feature/onedrive-token-fix
런타임 Python 3.11.9	상태 Marketplace 심사 대기		

목차 · TABLE OF CONTENTS

- | | | | |
|----|---------------------------------|----|------------------|
| 01 | 서비스 개요 & 사용자 여정 | 09 | 인증 & 보안 |
| 02 | 시스템 아키텍처 전체도 | 10 | Azure 리소스 인벤토리 |
| 03 | 기술 스택 상세 | 11 | 배포 & CI/CD |
| 04 | 백엔드 애플리케이션 구조 | 12 | 운영 · 스케줄러 · 로깅 |
| 05 | API 엔드포인트 레퍼런스 | 13 | 알려진 이슈 & 유지보수 노트 |
| 06 | AI 에이전트 파이프라인 (STEP 0-6) | 14 | 마켓플레이스 제출 체크리스트 |
| 07 | 데이터 아키텍처 (Cosmos·벡터·Blob·Queue) | 15 | 용어집 (쉬운 설명) |
| 08 | 외부 서비스 연동 | | |

이 문서를 읽는 법

본문 곳곳의  **쉬운 설명** 박스는 어려운 기술 용어를 일상어로 풀어 놓은 것입니다. 표의 상태 배지는 **구현완료** **진행중** **비활성/미연동** **주의** 로 표기합니다. 모든 내용은 실제 소스코드(`bybaek-backend`)를 직접 읽고 작성했으며, 파일·함수·라인은 `file.py:line` 형식으로 표기합니다.

보안 주의

이 문서에는 **API 키·시크릿·토큰 값은 일절 포함되어 있지 않습니다**. 리소스 *이름*과 *환경변수 이름*만 기재합니다. 실제 시크릿은 `.env` (로컬, git 미추적) 및 Azure App Service 구성/GitHub Secrets에만 보관하세요. 이 HTML/PDF를 외부에 공유해도 자격증명은 노출되지 않습니다.

01

서비스 개요 & 사용자 여정

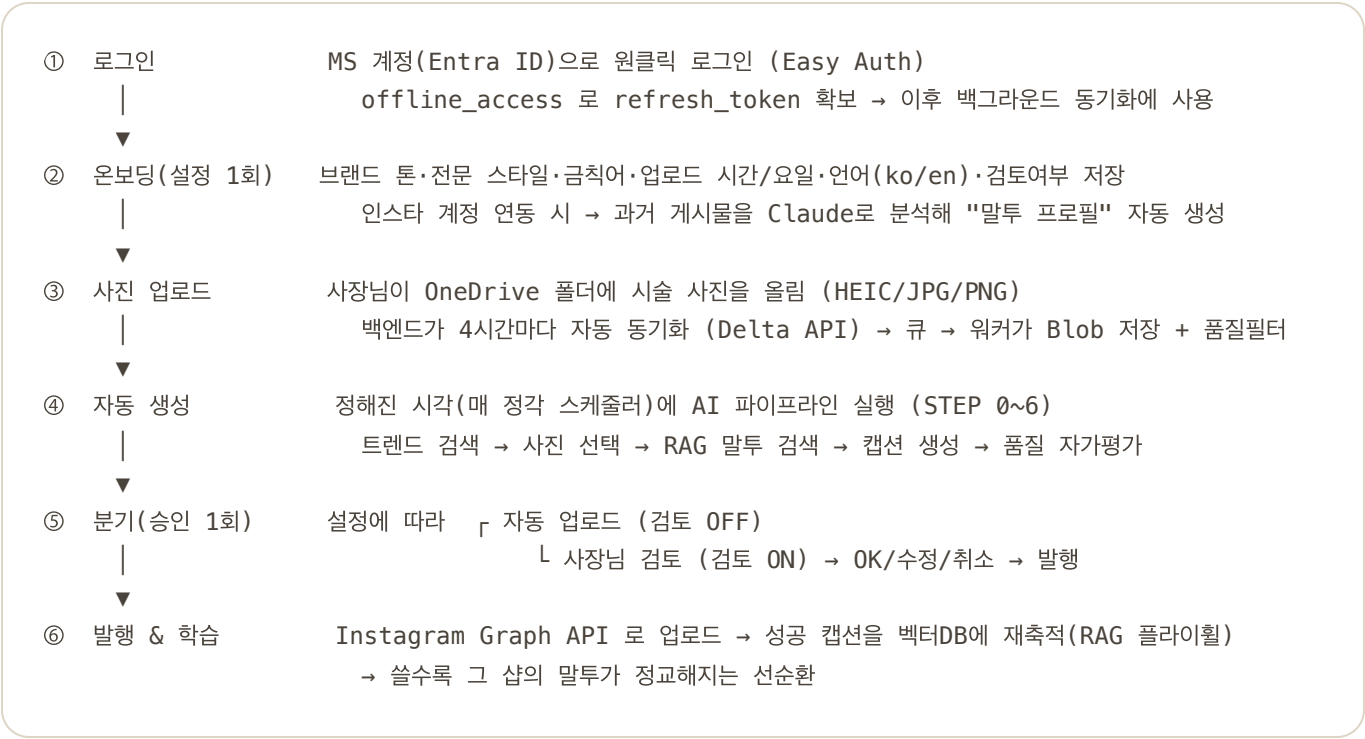
소상공인 바버샵 사장님의 인스타그램 마케팅을 AI로 완전 자동화하는 B2B SaaS입니다. 사장님은 **설정 1회 + 승인 1회**만 하면 됩니다.

사장님이 **OneDrive**에 **시술 사진**을 올려두면, 백엔드가 이를 자동으로 수집하고, 실시간 헤어 트렌드를 반영해 **인스타그램 게시물(캡션·해시태그·CTA)**을 자동 생성하고, 설정에 따라 **자동 업로드**하거나 **사장님 검토**를 거쳐 발행합니다.

쉬운 설명 — 이 백엔드가 하는 일 한 줄 요약

"사장님의 사진 폴더(OneDrive)를 지켜보다가, 요즘 유행을 검색해서 반영한 인스타 글을 대신 써주고, 정해진 시간에 올려주는 자동 SNS 담당 직원"입니다. 그 '직원'의 두뇌가 바로 AI 에이전트 파이프라인(6장)입니다.

핵심 사용자 여정 (End-to-End)



FRONTEND
bybaekofficial.com
Next.js · React · TypeScript

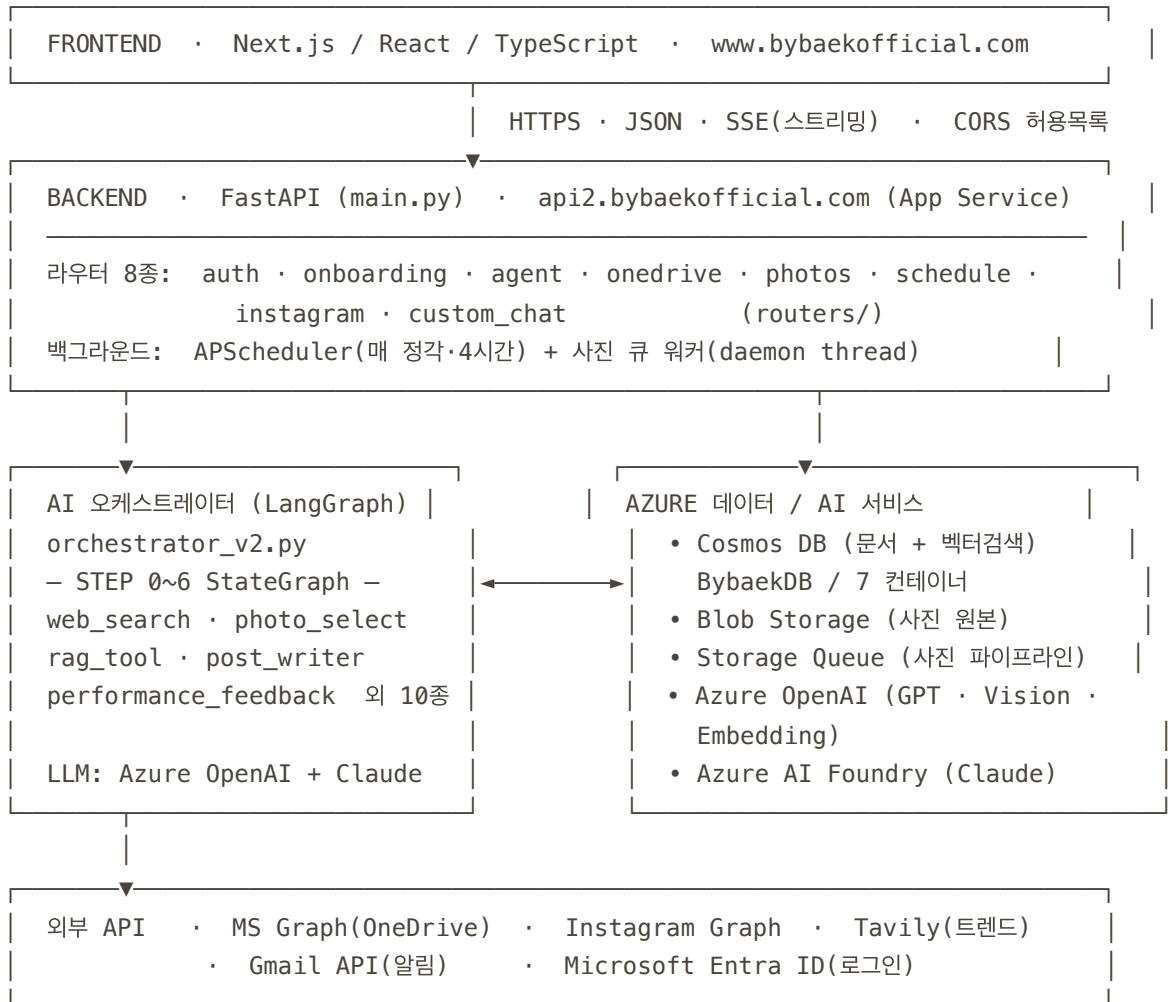
BACKEND (본 문서)
api2.bybaekofficial.com
FastAPI · Azure App Service

타겟 고객
소상공인 바버샵
B2B SaaS · Day-1 글로벌(ko/en)

02

시스템 아키텍처 전체도

프론트엔드 → FastAPI 백엔드 → (AI 오케스트레이터 · Azure 데이터/AI 서비스 · 외부 API)의 3계층 구조입니다.



💡 쉬운 설명 — "3계층"이란?

프론트엔드는 사장님이 보는 화면(웹사이트), **백엔드**는 화면 뒤에서 실제 일을 처리하는 서버(이 문서의 대상), **데이터/AI/외부 API**는 백엔드가 필요할 때 불러 쓰는 참고와 도구들입니다. 백엔드가 가운데서 모든 것을 지휘합니다.

요청 흐름 두 가지

- **동기(실시간) 흐름** — 사장님이 화면에서 버튼을 누름 → 라우터가 즉시 처리해 응답. 예: 로그인, 온보딩 저장, 사진 조회, **custom_chat** 실시간 캡션 스트리밍.
- **비동기(백그라운드) 흐름** — 사람이 없어도 서버가 스스로 실행. 예: 매 정각 스케줄러의 자동 게시물 생성, 4시간마다 OneDrive 동기화, 큐 워커의 사진 다운로드·필터링.

기술 스택 상세

Python 3.11 + FastAPI를 중심으로, AI 오케스트레이션은 LangGraph·Semantic Kernel, LLM은 Azure OpenAI와 Claude(Azure AI Foundry)를 함께 사용합니다.

분류	기술 / 패키지	버전(REQUIREMENTS)	용도
웹 프레임 워크	fastapi	0.115.0	비동기 REST API 서버
ASGI 서버	uvicorn[standard] · gunicorn	0.30.0	프로덕션 실행(워커)
AI 그래프	langgraph	최신	STEP 0~6 상태그래프 오케스트레이션
AI 커널	semantic-kernel[azure]	≥1.30.0	Azure OpenAI 호출·평가(judge)
LLM SDK	openai	≥1.98.0	GPT 채팅 · Vision · Embedding
LLM SDK	anthropic	≥0.49.0	Claude 생성·분석·스트리밍
데이터베이스	azure-cosmos	4.7.0	NoSQL 문서 + 벡터 검색
스토리지	azure-storage-blob	12.21.0	사진 원본 저장 · SAS URL
큐	azure-storage-queue	≥12.0.0	사진 동기화 비동기 처리
인증 (Azure)	azure-identity	≥1.15.0	Managed Identity / AAD 토큰
인증 (Entra)	msal	≥1.30.0	OneDrive refresh_token 갱신
트렌드 검색	tavily-python	0.3.3	실시간 웹 검색
이미지 처리	pillow · pillow-heif · opencv-python-headless · numpy	10.4.0 ...	HEIC→JPG 변환 · 블러/밝기 분석
스케줄러	apscheduler	최신	매 정각/4시간 크론 잡
데이터 검증	pydantic[email]	≥2.10,<3	요청/응답 스키마

분류	기술 / 패키지	버전(REQUIREMENTS)	용도
HTTP 클라이언트	httpx	0.27.0	Graph·Instagram·Blob 호출
토큰/JWT	python-jose	3.3.0	토큰 처리

참고 — 사용되지 않는 잔재

`azure-search-documents` 는 과거 Azure AI Search 기반 RAG의 잔재로 `requirements.txt` 에 남아 있으나, 현재 벡터 검색은 **Cosmos DB로 통합**되어 실사용 코드 경로에서는 호출되지 않습니다(13장 참조).

04 백엔드 애플리케이션 구조

`main.py` 가 FastAPI 앱을 만들고, 라우터 8종을 등록하며, 시작 시(`lifespan`) 스케줄러와 큐 워커를 겁니다.

디렉터리 구조

```
# 핵심만 발췌
bybaek-backend/
├─ main.py                # FastAPI 엔트리포인트 · 라우터 등록 · 스케줄러/워커 기동
├─ orchestrator_v2.py     # AI 파이프라인 (LangGraph StateGraph, STEP 0~6)
├─ agents/                # AI 에이전트 10종 + 레거시 오케스트레이터
│   ├── web_search.py · competitor_analysis.py
│   ├── photo_filter.py · photo_select.py · photo_feedback.py
│   ├── post_writer.py · rag_tool.py · performance_feedback.py
│   └── insta_analyzer.py · orchestrator.py (레거시 v1)
├─ routers/               # API 엔드포인트 8종
│   ├── auth.py · onboarding.py · agent.py · onedrive.py
│   ├── photos.py · schedule.py · instagram.py · custom_chat.py
├─ services/              # 데이터/외부 서비스 계층
│   ├── cosmos_client.py · cosmos_db.py · vector_db.py
│   └── blob_storage.py · email_service.py
├─ workers/
│   └── photo_queue_worker.py # 사진 큐 소비 워커 (daemon thread)
├─ utils/
│   ├── claudia_auth.py    # Claude(Foundry) AAD/Managed Identity 인증
│   └── logging.py
├─ prompts/               # 프롬프트 템플릿 (post_writer / custom_chat / examples)
├─ auth/appService_auth_check.py # Easy Auth 세션/헤더 확인 헬퍼
├─ seed_embeddings.py      # Cold Start용 seed 캡션 벡터 주입
├─ requirements.txt · .env(.example) · .python-version(3.11.9)
└─ .github/workflows/main_bybaek-b.yml # GitHub Actions 배포
```

애플리케이션 부팅 순서 **main.py**

1. `load_dotenv()` — 환경변수 로드.
2. `FastAPI(title="BYBAEK API", version="1.0.0", lifespan=lifespan)` 생성.
3. **CORS** 미들웨어 — `localhost:3000` + 환경변수 `FRONTEND_URL` (кома 구분 다중) 허용, `credentials` 허용.
4. 라우터 8종 등록(각 `/api/*` prefix).
5. `lifespan` 진입 시: `APScheduler`에 잡 2개 등록 후 `scheduler.start()`, 이어서 `start_worker()` 로 큐 워커 daemon thread 기동.
6. `GET /` 헬스체크: `{"status":"ok","service":"BYBAEK API"}` (App Service 배포 확인용).

백그라운드 잡	주기 (KST)	함수	동작
자동 게시물 생성	매 정각 (minute=0)	<code>_check_and_run_schedules</code>	모든 샷을 순회하며 업로드 시각·요일 · <code>insta_auto_upload_yn=Y</code> 매칭 시 <code>run_pipeline(trigger="auto")</code>
OneDrive 동기화	4시간마다 (hour=*/4)	<code>_sync_all_shops_onedrive</code>	모든 샷의 <code>sync_photos_internal()</code> 호출 (refresh_token 기반)
사진 큐 워커	상시 (30초 폴링)	<code>start_worker</code> → <code>polling_loop</code>	Storage Queue 소비 → 다운로드 → Blob 저장 → 필터 트리거

💡 쉬운 설명 — LIFESPAN / 스케줄러 / 워커

lifespan은 "서버가 켜질 때 한 번, 꺼질 때 한 번 실행되는 준비/정리 코드"입니다. 여기서 **스케줄러**(정해진 시간에 알람처럼 실행)와 **워커**(할 일 목록을 계속 지켜보며 처리하는 백그라운드 일꾼)를 겁니다. 덕분에 사장님이 접속하지 않아도 서버가 알아서 사진을 모으고 글을 올립니다.

스케줄 판정 로직 (MAIN.PY)

업로드 시각은 DB의 `insta_upload_time` (예: "07:00 PM")을 24시간제 정각으로 변환해 현재 시각과 비교합니다. 요일은 `insta_upload_days` (사용자 지정)를 우선 적용하고, 비어 있으면 `insta_upload_time_slot` ("매일"/"주 3회"→월·수·금/"주 1회"→월) 기본값으로 풀백합니다.

05

API 엔드포인트 레퍼런스

전 라우터의 엔드포인트 전체 목록입니다. 경로는 `main.py`의 prefix를 포함한 최종 경로입니다.

Auth **/api/auth**

메서드	경로	설명
GET	/ms/callback	MS 로그인 후 프론트 /auth/callback 로 리다이렉트
GET	/instagram?code=...	인스타 OAuth: 단기토큰 교환 → 장기토큰 교환 → 저장 → 과거글 분석 (BackgroundTask)
GET	/status/{shop_id}	{is_onedrive_connected, is_insta_connected}
GET	/me	Easy Auth 헤더로 신원 확인 · refresh_token 저장 · {shop_id, is_new}

Onboarding **/api/onboarding**

메서드	경로	설명
POST	/reference	레퍼런스(예시) 사진 앨범 저장 → 사진필터 Stage2 few-shot에 사용
POST	/shop_id	온보딩(브랜드톤·스타일·금칙어·스케줄·언어 등) 저장
GET	/shop_id	온보딩 조회
POST	/shop_id/reanalyze	인스타 말투 프로필 재분석 수동 트리거

Agent **/api/agent**

메서드	경로	설명
POST	/run	파이프라인 실행 {shop_id, trigger, photo_ids?, message?}
POST	/review	검토 처리 action = ok edit cancel → 업로드/취소
POST	/save	초안 수동 저장
GET	/posts/{shop_id}	게시물 목록
GET	/post/detail/{post_id}	게시물 상세
GET	/metrics/{shop_id}	caption_score 평균·retry율·모델분포·점수분포 집계

OneDrive /api/onedrive

메서드	경로	설명
POST	/sync-photos	Graph Delta API 로 변경분만 수집 → 큐로 배치 전송(10장 단위) · <code>x-ms-token-aad-access-token</code> 헤더 필요

Photos /api/photos

메서드	경로	설명
GET	/all/{shop_id}	사용가능 사진 목록(SAS 서명 URL)
GET	/albums/{shop_id}	앨범 목록(썸네일 SAS)
GET	/albums/{shop_id}/{album_id}	앨범 내 사진
POST	/albums	앨범 생성
POST	/filter	AI 사진 필터 실행(백그라운드, 10장 청크)
GET	/status/{shop_id}	필터 진행 상태 (passed/failed/pending)
GET·HEAD	/proxy/{shop_id}/{photo_id}/image.jpg	인스타용 이미지 프록시(HEAD는 검증 통과용 가짜 헤더)
DELETE	/albums/{shop_id}/{album_id} · /{shop_id}/{photo_id}	앨범/사진 삭제
POST	/filter/test/{shop_id}	동기 디버그: 최대 3장 필터 후 원시 결과 반환

메서드	경로	설명
GET	/api/schedule/{shop_id}	업로드 시각·다음 실행·타임존
POST	/api/schedule/{shop_id}/update	업로드 시각 변경
POST	/api/instagram/upload	인스타 발행(단일/캐러셀) · 종횡비 정규화 후 Graph 발행 · {media_id}
POST	/api/custom_chat/manual_chat	Claude 실시간 스트리밍 캡션 생성 (SSE, text/event-stream)
GET	/	헬스체크

인증 모델 주의

라우트에 FastAPI Depends() 가드가 없습니다. 신원은 App Service Easy Auth가 주입하는 헤더 (X-MS-CLIENT-PRINCIPAL-ID/NAME , x-ms-token-aad-access-token/-refresh-token)로 흐르며, shop_id = MS principal ID 입니다. auth.py 에는 개발용 하드코딩 폴백 (test_barber_jiyeon)이 있어 운영 배포 전 제거 검토가 필요합니다(13장).

06 AI 에이전트 파이프라인 (STEP 0-6)

핵심 두뇌입니다. orchestrator_v2.py 의 LangGraph StateGraph가 상태(PostState)를 노드에서 노드로 넘기며, 두 개의 품질 게이트(트렌드.캡션)와 재시도.모델 승격.사람 검토 분기를 수행합니다.

💡 쉬운 설명 — STATEGRAPH / 노드 / 게이트

공장의 컨베이어벨트를 떠올리세요. 노드는 각 작업 공정, 상태(PostState)는 벨트 위를 흐르는 작업물 상자입니다. 게이트는 "품질검사대"로, 기준(0.7점) 미달이면 다시 앞 공정으로 돌려보내(재시도) 다시 만들게 합니다. 사람(LangGraph)이 아니라 코드가 이 흐름을 자동으로 지휘합니다.

전역 상수 & 모델 티어

QUALITY_THRESHOLD

0.7

트렌드.캡션 품질 통과 기준점

MAX_RETRY

2

각 게이트의 최대 재시도 횟수

MIN_PHOTO_COUNT

1

최소 선택 사진 수(미달 시 기간 확장)

모델 티어는 `_get_deployment_name(tier)` 가 환경변수로 해석합니다.

`mini` → `AZURE_OPENAI_DEPLOYMENT_MINI` , `full` → `AZURE_OPENAI_DEPLOYMENT_FULL` . 비용을 아끼기 위해 기본 `mini` 로 시작하고, 필요 시 `full` 로 승격합니다.

STEP 0 → 6 상세 흐름



단계별 판정 로직

STEP	노드 함수	핵심 판정 / 라우팅
0	node_classify	manual 또는 photo_ids≥5 → full, 아니면 mini
1	node_fetch_data	4개 소스 병렬 수집(<code>_timed</code> 로 계속)
1.5	node_fetch_performance	초안 ≥3개면 top⅓ vs bottom⅓ 패턴 비교
2	route_after_trend_eval	<code>score<0.7 && retries<2</code> → <code>retry_trend</code> , 아니면 <code>select_photos</code>
3	node_select_photos	manual+ids면 직접 조회, 아니면 director 패턴 선택
4	node_search_rag	타입별 벡터검색 + GPT 압축 + 성과 인사이트 병합
5	route_after_write	≥0.7→save / retries<2→retry / mini소진→승격 / full소진→수용
6	node_save_draft	<code>insta_review_bfr_upload_yn</code> 로 자동발행 vs 검토 분기

에이전트 10종 카탈로그

에이전트	단계	모델	역할	상태
web_search	STEP 1	GPT-mini + Tavily	날씨·헤어트렌드·프로모 수집 (일일 캐시, 4단계 필터)	사용
competitor_analysis	STEP 1 내부	GPT-mini + Tavily	경쟁샵 스타일/해시태그·차별화 갭 인사이트	사용
photo_filter	파이프라인 前(워커)	OpenCV + GPT Vision	2단계 사진 QC(블러/밝기 → Vision 5항목 25점)	사용
photo_select	STEP 3	규칙 + Claude(확 장)	페이드2+스타일1+분위기1, 14 일 쿨다운	사용
rag_tool	STEP 4	Embedding + GPT-mini	벡터검색·압축 + 발행글 인덱싱 (플라이휠)	사용
performance_feedback	STEP 1.5/4	순수 Python	과거 점수 분석 → 성과 인사이 트 주입	사용
post_writer	STEP 5	Claude Sonnet 4.6	캡션·해시태그·CTA 생성 + 할 루시네이션 검증	사용
insta_analyzer	온보딩(파이 프라인 外)	Claude + Embedding	과거 인스타그램 말투 프로필화 + RAG 콜드스타트 백필	사용
photo_feedback	(미배선)	GPT-mini	사진 탈락사유 학습(정의됨, 호 출부 없음)	비활성
orchestrator (v1)	(레거시)	—	LangGraph 이전의 선형 버전 (참조용)	레거시

품질 게이트 — LLM-as-Judge

캡션 점수는 Azure GPT가 5개 항목에 가중 평균으로 채점합니다(`_evaluate_caption`):

항목	가중치	의미
reservation_inquiry	40%	예약/DM 문의로 이어질 가능성
fade_keyword	25%	페이드컷 등 핵심 키워드 포함
cta_strength	15%	행동유도(CTA) 강도
brand_tone	10%	브랜드 톤 일치
target_appeal	10%	20-40대 남성 공감도

할루시네이션 3중 제어

1. 프롬프트 가드레일 — `prompts/post_writer/system.md` + `examples/bad_captions.md` (9개 안티예시)에 "경력 연수·예약 현황·과장·할인 금지" 명시.
2. 후처리 정규식 — `_validate_and_clean` 이 `\d+년 경력` , `\d+자리 남` , "마감 임박", "오늘만 할인" 패턴 감지(단, `shop_intro` 에 실제로 있으면 예외).
3. 피드백 재시도 — 감지 시 `needs_retry` 플래그로 해당 문제를 명시해 1회 재생성.

RAG 플라이휠 (자가 학습 루프)

업로드 성공한 캡션을 `index_post_for_rag` 가 `caption_body/hashtag_set/cta` 로 나눠 벡터DB에 재축적합니다(composite-key). 신규 샵은 `seed_embeddings.py` 의 seed 캡션과 `insta_analyzer` 의 과거글 백필로 콜드스타트를 해결합니다. 쓸수록 그 샵의 말투가 정교해집니다.

데이터 아키텍처

단일 Cosmos DB **BybaekDB** 에 운영 컨테이너 6종 + 벡터 컨테이너 1종. 모든 컨테이너는 **shop_id** 로 파티셔닝됩니다. 사진 원본은 Blob, 비동기 처리는 Storage Queue를 씁니다.

💡 쉬운 설명 — 컨테이너 / 파티션 키 / 벡터 검색

컨테이너는 엑셀의 '시트'처럼 같은 종류 데이터를 모아두는 표입니다. **파티션 키(shop_id)**는 "가게 번호별로 서랍을 나눠 넣는 기준"이라, 한 가게 데이터를 빠르고 싸게 찾게 해줍니다. **벡터 검색**은 글을 숫자 좌표(임베딩)로 바꿔 "의미가 비슷한 글"을 찾는 방식으로, 그 가게의 말투와 닮은 과거 캡션을 골라옵니다.

컨테이너	문서 ID	주요 필드	비고
Shop	= shop_id	브랜드설정 전체 · 스케줄 · 연동플래그 · 인스타/원드라이브 OAuth 토큰 · insta_style_profile · one_delta_link	온보딩+인증 통합
Photo	= photo_id	blob_url (SAS 제거) · filter_status · is_usable · fade_cut_score · detected_angle · style_tags · used_at	필터 결과 보존
Album	= album_id	album_name · photo_ids[] · 썸네일	레퍼런스 앨범 reference_{shop_id}
Post	post_{uuid}	caption · hashtags[] · status · review_action · caption_score · model_used · metrics{} · review_deadline (+24h)	초안·발행 이력
Cache	{shop_id}_{date}	result{} · ttl=86400 · expire_at	유일하게 TTL(24h) 사용 — 웹서치 캐시
RejectionLog	임의	사진 탈락 로그(최신 50)	photo_feedback용(현재 비활성)
RagVectors	{post_id}_{type}	caption · caption_vector (1536차원) · content_type	벡터 검색 전용

벡터 검색 상세

services/vector_db.py

임베딩 모델

text-embedding-3-small

1536차원 · Azure OpenAI

거리 함수

VectorDistance()

코사인 유사도 · ORDER BY 오름차순

content_type

4종

caption_body · hashtag_set · cta · structure

쿼리 범위

단일 파티션

shop_id 파티션만 조회(RU 절약) · top_k 최대 50

Blob Storage & Queue

항목	값	설명
스토리지 계정	bybaekstore1	코드 일부에 하드코딩(13장 참조)
Blob 컨테이너	env: AZURE_BLOB_CONTAINER_NAME	사진 원본
Blob 경로 규칙	{container}/{shop_id}/{md5(경로)}.{ext}	HEIC/HEIF → .jpg 강제 변환
접근 방식	읽기 전용 SAS (기본 2h)	비공개 Blob에 시간제한 서명 URL
큐 이름	bybaek-photo-sync	사진 동기화 배치 메시지
큐 폴링	30초 · 10건/폴 · visibility 300s	실패 메시지는 재노출로 재시도

워커 처리 흐름 `workers/photo_queue_worker.py`

```
Queue(bybaek-photo-sync) —30초 폴링→ receive(10건)
| 메시지 = {shop_id, drive_id, token, container_name, photos[]}
▼
process_message: 각 사진마다
| photo_id = "photo_{shop_id}_{md5(relative_path)}"      (중복 방지)
| DB 존재 확인(get_photo_by_id) → 있으면 skip
| Graph에서 다운로드: GET /drives/{drive_id}/items/{item_id}/content
| HEIC/HEIF → JPG 변환(pillow-heif, q95)
| Blob 업로드: {shop_id}/{md5}.jpg
|   ↳ Cosmos save_photo + filter_list에 2h SAS URL 추가
▼
성공 메시지 delete → shop별로 trigger_filter(사진필터 실행)
```

이메일 알림 `services/email_service.py`

SMTP가 아니라 **Gmail API(OAuth)**를 사용합니다. 스코프 `gmail.send`, 토큰은

`services/token.json` · `credentials.json`. 새 초안이 준비되면 `send_draft_notification` 이

"[ByBaek] 새 게시물 초안이 준비됐어요 ✂ " 메일을 사장님에게 발송합니다.

08

외부 서비스 연동

OneDrive(MS Graph), Instagram Graph, Claude(Azure AI Foundry), Tavily, Gmail 등 5+개 외부 서비스와 실연동됩니다.

서비스	용도	엔드포인트/버전	인증
MS Graph (OneDrive)	사진 자동 동기 화(Delta)	graph.microsoft.com/v1.0	Easy Auth 토큰 + refresh_token(MSAL 갱신)
Instagram Graph	게시물 발행(단 일/캐러셀)	graph.instagram.com/v25.0	장기 access_token(Cosmos 저 장)
Instagram OAuth	계정 연동	api.instagram.com/oauth	client_id/secret + code 교환
Claude	캡션 생성·말투 분석·챗	bybaek-foundry.../anthropic	AAD Bearer(Managed Identity)
Azure OpenAI	평가·요약 ·Vision·임베딩	bybaek.openai.azure.com	API Key
Tavily	실시간 트렌드 검색	Tavily API	API Key
Gmail API	초안 알림 메일	googleapis.com/gmail	OAuth(token.json)

OneDrive Delta 동기화 routers/onedrive.py

처음엔 전체 목록을, 이후엔 **변경분만**(@odata.deltaLink 저장·재사용) 가져옵니다. 사진만 필터링해 10장 단위로 큐에 넣고, 스케줄러 경로(sync_photos_internal)에서는 요청 헤더가 없으므로 저장된 refresh_token 으로 MSAL이 새 Graph 토큰을 발급합니다(offline_access 스코프 필요).

💡 쉬운 설명 — DELTA API

매번 폴더 전체를 다시 훑으면 느리고 비쌉니다. Delta는 "지난번 이후로 새로 바뀐 것만 알려줘"라고 요청하는 방식이라, 사진 몇 장 추가돼도 그 몇 장만 빠르게 가져옵니다.

Instagram 발행 routers/instagram.py

이미지 중횡비를 인스타 허용 범위(0.8 ~ 1.91)로 자동 크롭 정규화 → 컨테이너 생성 → status_code=FINISHED 까지 폴링(10회×2초) → 발행. 다중 이미지는 캐러셀(최대 10장)로 처리하고, 임시 크롭본은 temp_cropped/ 에서 발행 후 정리합니다.

Claude 실시간 챗 routers/custom_chat.py

브랜드설정 + 실시간 트렌드를 합친 프롬프트로 claude-sonnet-4-6 를 스트리밍 호출(SSE)합니다. 토큰 단위로 프론트에 즉시 전달되어 "타이핑되는 것처럼" 캡션이 나타납니다.

09 인증 & 보안

사용자 로그인은 Entra ID + App Service Easy Auth, 서버-Claude 간은 Managed Identity(AAD 토큰), 외부 SNS는 OAuth 장기토큰으로 3원화되어 있습니다.

3가지 인증 경로

대상	방식	토큰 저장/획득
사장님 로그인	Entra ID + Easy Auth	App Service가 헤더 주입 · refresh_token 은 Cosmos 저장
OneDrive(Graph)	OAuth + MSAL	스케줄러 시점에 refresh_token으로 재발급
Instagram	OAuth 장기토큰	단기→장기 교환 후 Cosmos insta_access_token
Claude(Foundry)	AAD Bearer(MI)	get_claude_token() 자동갱신
Azure OpenAI / Tavily	API Key	환경변수

Claude 인증의 함정 `utils/claude_auth.py`

이 프로젝트의 대표적 장애 원인 (해결됨)

Foundry의 `/anthropic` 엔드포인트는 **api-key**를 거부하고 **AAD Bearer** 토큰만 받으며, 토큰 audience(scope)가 정확히 `https://ai.azure.com/.default` 여야 합니다. 이를 어기면 401이 나고 `post_writer` 가 **목업(가짜) 캡션**으로 폴백합니다. 또한 이 앱에는 OneDrive용 `AZURE_CLIENT_ID/TENANT_ID/CLIENT_SECRET` 가 있어 `DefaultAzureCredential` 이 시스템 Managed Identity를 못 찾고 실패하므로, **명시적 체인** `ChainedTokenCredential(ManagedIdentityCredential(), AzureCliCredential())` 을 사용합니다(배포=MI, 로컬=az login).

운영 조건: App Service의 시스템 할당 **Managed Identity**에 `bybaek-foundry` 의 **Cognitive Services User** 역할이 부여되어 있어야 합니다.

💡 쉬운 설명 — MANAGED IDENTITY

서버가 다른 Azure 서비스에 접속할 때, 비밀번호(키)를 코드에 넣는 대신 Azure가 서버에게 발급하는 "사원 증"입니다. 키가 유출될 위험이 없고, Azure가 알아서 갱신해 줍니다.

보안 점검 요약

- **양호** `.env` · `credentials.json` · `token.json` 은 `.gitignore` 에 포함되어 git에 추적되지 않음(확인 완료).
- **양호** Claude는 키가 아닌 Managed Identity 토큰 사용.
- **주의** 일부 Cosmos 쿼리가 `shop_id` 를 f-string으로 SQL에 삽입 → 파라미터 바인딩 권장(13장).
- **주의** `auth.py` 의 개발용 하드코딩 폴백(`test_barber_jiyeon`) → 운영 제거 검토.
- **확인** Blob 공개/비공개 정책 — 현재 발행 시 SAS 제거한 직접 URL 사용(공개 전제). Marketplace 기준에 맞춰 재점검 권장.

Azure 리소스 인벤토리

현재 개인 Azure 구독(리소스그룹 `rg-bybaek`, 리전 Korea Central)에 배포되어 있습니다. 아래는 코드·설정에서 확인된 리소스 이름입니다.

시크릿 없음

아래 표에는 리소스 이름과 환경변수 이름만 있으며 키/토큰 값은 없습니다. 실제 값은 Azure Portal의 각 리소스 · App Service 구성 · GitHub Secrets에서 관리하세요.

리소스 종류	이름 / 식별자	연결 환경변수	용도
App Service (Backend)	bybaek-backend	—	FastAPI 호스팅 · api2.bybaekofficial.com
App Service (Frontend)	bybaek-frontend	FRONTEND_URL	Next.js · www.bybaekofficial.com
Cosmos DB 계정	bybaek-db	AZURE_COSMOS_URL / _KEY	문서 + 벡터 (DB: BybaekDB)
Storage 계정	bybaekstore1	AZURE_STORAGE_CONNECTION_STRING	Blob(사진) + Queue
Blob 컨테이너	(env값)	AZURE_BLOB_CONTAINER_NAME	사진 원본
Storage Queue	bybaek-photo-sync	(동일 연결문자열)	사진 동기화 배치
Azure OpenAI	bybaek	AZURE_OPENAI_ENDPOINT / _KEY	GPT · Vision · Embedding
OpenAI 배포(mini)	env값	AZURE_OPENAI_DEPLOYMENT_MINI	기본 티어
OpenAI 배포(full)	env값	AZURE_OPENAI_DEPLOYMENT_FULL	승격 티어
임베딩 배포	text-embedding-3-small	AZURE_OPENAI_EMBEDDING_DEPLOYMENT	RAG 벡터화
Azure AI Foundry	bybaek-foundry	AZURE_CLAUDE_ENDPOINT	Claude Sonnet 4.6 (/anthropic)
Entra ID 앱 등록	(client id)	AZURE_TENANT_ID / _CLIENT_ID / _CLIENT_SECRET	OneDrive/Graph · 로그인

모델 명칭에 관한 정확성 노트

코드의 Claude 호출은 `model="claude-sonnet-4-6"` 로 고정되어 있습니다. Azure OpenAI 배포 이름은 **환경변수로 주입**되어 유동적입니다(로컬 `.env` 에는 `gpt-4.1` 계열, README에는 구독 이전 후 `gpt-5-mini` 로 기재). 실제 운영 배포에서 어떤 배포명이 매핑돼 있는지는 **App Service 구성**에서 확인하는 것이 정확합니다.

11 배포 & CI/CD

`main` 브랜치에 push하면 GitHub Actions가 자동으로 App Service에 배포합니다. 배포 대상은 변수로 파라미터화되어 있어, 구독/앱 이전 시 값 두 개만 바꾸면 됩니다.

GitHub Actions 워크플로 `.github/workflows/main_bybaek-b.yml`

```
# 트리거: main push 또는 수동(workflow_dispatch)
1. Checkout (actions/checkout@v4)
2. Python 3.11 설정 (actions/setup-python@v5)
3. pip install -r requirements.txt # 의존성 설치 검증
4. azure/webapps-deploy@v3
   app-name:      ${vars.AZURE_WEBAPP_NAME} # 현재: bybaek-backend
   publish-profile: ${secrets.AZURE_WEBAPP_PUBLISH_PROFILE}
   package: .
```

💡 쉬운 설명 — CI/CD와 게시 프로필

CI/CD는 "코드를 올리면(push) 자동으로 서버에 반영되는" 자동 배포 파이프라인입니다. **게시 프로필(publish profile)**은 특정 App Service에 배포할 권한이 담긴 자격증명 파일로, GitHub Secrets에 안전하게 보관됩니다.

다른 구독/앱으로 이전하는 법

- GitHub → Settings → **Variables**: `AZURE_WEBAPP_NAME` 을 새 App Service 이름으로 변경.
- GitHub → Settings → **Secrets**: `AZURE_WEBAPP_PUBLISH_PROFILE` 을 새 앱의 게시 프로필(XML)로 교체.
- 새 App Service의 구성(환경변수)에 `.env.example` 항목을 모두 설정.
- 새 App Service에 시스템 **Managed Identity** 켜고 Foundry에 **Cognitive Services User** 역할 부여(Claude용).
- 커스텀 도메인·CORS(`FRONTEND_URL`) 재설정.

배포 이력 참고 (README 주석)

과거 구독의 URL은 `bybaek-b-...koreacentral-01.azurewebsites.net` 형태였고, 현재는 개인 구독 `rg-bybaek` (Korea Central)로 **무중단 이전** 완료. 복귀가 필요하면 위 변수 두 개와 환경변수를 이전 값으로 되돌리면 됩니다.

12

운영 · 스케줄러 · 로깅

정기 실행 (KST 기준)

작업	주기	영향
자동 게시물 생성	매 정각	업로드 시각·요일 매칭 샵 파이프라인 실행
OneDrive 동기화	4시간마다	새 사진 수집 → 큐 → 워커
큐 워커 폴링	30초	사진 다운로드/변환/저장/필터
웹서치 캐시	TTL 24h	중복 트렌드 검색 차단(비용 절감)

로깅 & 관측

- 파이프라인 각 단계는 `_timed()` 로 `[TIMING]` 단계 → `n.nns` 로그를 남깁니다(병목 진단용).
- 노드마다 `[orchestrator_v2]` ... 상태 로그 출력(tier, score, retries).
- `utils/logging.py` 가 `logs/app.log` + 콘솔 이중 출력. App Service에서는 **Log stream**으로 실시간 확인 가능.
- 운영 지표는 `GET /api/agent/metrics/{shop_id}` 에서 `caption_score` 평균·retry율·모델분포·점수분포로 확인.

흔한 장애와 대응

증상	원인	대응
캡션이 어색한 목업으로 나옴	Claude 401(잘못된 scope/MI 역할)	MI에 Cognitive Services User 부여 · scope <code>ai.azure.com</code> 확인(9장)
OneDrive 동기화 실패	refresh_token 없음/만료	사장님 재로그인(offline_access) 유도
인스타 발행 실패	중횡비/토큰 만료/미디어 상태	정규화 로그·장기토큰 갱신 확인
사진이 안 들어옴	큐 워커 미기동/Graph 토큰	워커 로그·큐 적체 확인
트렌드 품질 낮음 반복	Tavily 결과 빈약	재시도 2회 후 진행(정상 동작)

13

알려진 이슈 & 유지보수 노트

코드를 직접 읽으며 발견한, 수익화·마켓플레이스 전에 알아두면 좋은 항목들입니다. 대부분 즉시 장애는 아니지만 기술 부채/리스크로 관리 대상입니다.

등급	항목	위치
보안	SQL 인젝션 표면: <code>shop_id</code> 를 f-string으로 쿼리에 삽입	cosmos (<code>get_</code> <code>get_re</code>
보안	개발용 하드코딩 신원 폴백 <code>test_barber_jiyeon</code>	router
일관성	스토리지 계정 <code>bybaekstore1</code> 이 코드 여러 곳에 하드코딩	worker
일관성	모델명 불일치: README(gpt-5-mini) vs .env(gpt-4.1) vs 코드(claude-sonnet-4-6)	README
잔재	Azure AI Search 관련 의존성/환경변수(벡터는 Cosmos로 이전됨)	require seed_e
미배선	<code>photo_feedback</code> (사진 탈락 학습) 정의됐으나 호출부 없음	agents
빈 파일	<code>custom_chat/system.md</code> · <code>post_writer/user_template.md</code> · <code>good_captions.md</code> 비어 있음	prompt
비용/성능	Cosmos <code>get_cosmos_container</code> 가 호출마다 새 클라이언트 생성(캐시 없음)	service

진행 중 로드맵 (README 기준)

Instagram 자동 발행(Meta 앱 심사) · Claude 글쓰기/분석 정식 활성화(Foundry 쿼터 승인 대기) · Hybrid Search(BM25+Vector) 도입 · Microsoft Azure Marketplace 심사/런칭.

14

마켓플레이스 제출 체크리스트

누락 없는 제출·수익화를 위해 기술 관점에서 점검할 항목입니다. (상업 조건·과금 연동 등 비기술 항목은 파트너 IUI와 별도 확인.)

기술 준비도

- **완료** 헬스체크 엔드포인트(`GET /`) — App Service 가용성 프로브.
- **완료** CI/CD 파라미터화 — 구독 이전 대응.
- **완료** Managed Identity 기반 Claude 인증 — 키 유출 리스크 제거.
- **점검** 시크릿 전량 App Service 구성/Key Vault로 이관 여부(로컬 .env 의존 제거).
- **점검** Blob 공개/비공개 정책 확정 및 발행 URL 방식 문서화.
- **권장** SQL 파라미터 바인딩 · 하드코딩 폴백 제거(13장) 선반영.
- **점검** 개인정보/데이터 처리 방침(사진·인스타 토큰 보관) 문서화.
- **점검** Meta 앱 심사(Instagram 자동 발행) 승인 상태.
- **점검** Foundry Claude 쿼터 승인 상태.

운영 준비도

- 장애 대응 런북(12장 표) 팀 공유.
- 로그/지표 모니터링 대시보드(caption_score, retry율, 큐 적체).
- 백업/복구: Cosmos 지속 백업 정책, Blob 수명주기 정책.
- 다국어(ko/en) 캡션 품질 샘플 검증.

협력 관계

Microsoft Korea — Azure 기술 미팅 · Marketplace 제출 완료, 심사/런칭 대기. **IUI** — Marketplace 등록 담당 · 출시 전 과정 공식 개발 협력사.

문서에 등장하는 기술 용어를 일상어로 풀었습니다.

용어	쉬운 설명
FastAPI	파이썬으로 API 서버를 빠르게 만드는 도구. 이 백엔드의 뼈대.
라우터(Router)	"이 주소로 오는 요청은 이 함수가 처리" 하고 길을 나눠주는 교통정리 담당.
엔드포인트(Endpoint)	요청을 받는 특정 주소(URL). 예: <code>/api/agent/run</code> .
오케스트레이터	여러 AI 에이전트를 순서대로/조건대로 지휘하는 지휘자.
LangGraph / StateGraph	AI 작업 흐름을 '그래프(순서도)'로 그려 자동 실행하는 도구.
에이전트(Agent)	한 가지 일(트렌드검색, 사진선택, 글쓰기 등)을 담당하는 작은 AI 일꾼.
RAG	AI가 글을 쓰기 전, 그 가게의 과거 캡션을 검색해 참고하는 방식. "찾아보고 쓰기".
임베딩(Embedding)	문장을 의미가 담긴 숫자 좌표로 바꾼 것. 비슷한 뜻이면 좌표도 가깝다.
벡터 검색	그 숫자 좌표로 '의미가 비슷한' 문장을 찾는 검색.
LLM-as-Judge	AI가 쓴 글을 또 다른 AI가 점수로 채점하는 자가 품질검사.
할루시네이션	AI가 사실이 아닌 것을 지어내는 현상(예: 없는 경력 언급).
티어(mini/full)	싼 모델(mini)로 먼저 시도하고, 품질 미달 시 좋은 모델(full)로 올리는 비용 전략.
Human-in-the-loop	자동화 중간에 사람이 최종 승인하는 안전장치. 여기서 '검토 후 발행'.
Cosmos DB	Azure의 NoSQL 데이터베이스. 이 서비스의 모든 데이터가 여기 저장됨.
컨테이너 / 파티션 키	데이터를 담는 표 / 가게(shop_id)별로 나눠 담는 기준.
Blob Storage	사진 같은 큰 파일을 저장하는 Azure 창고.
SAS URL	비공개 파일에 '시간 제한 출입증'을 붙인 임시 링크.
Queue(큐)	"할 일 목록". 사진 다운로드 작업을 줄 세워두면 워커가 순서대로 처리.
워커(Worker)	큐의 할 일을 계속 지켜보며 처리하는 백그라운드 일꾼.
스케줄러	정해진 시간에 알람처럼 코드를 실행하는 도구(매 정각/4시간).
Delta API	지난번 이후 '바뀐 것만' 가져오는 효율적 동기화 방식.

용어	쉬운 설명
OAuth / refresh_token	외부 서비스 로그인 표준 / 로그인 상태를 갱신하는 재발급용 열쇠.
Managed Identity	키 없이 Azure가 서버에 발급하는 '사원증' 인증.
Easy Auth	App Service가 로그인 처리를 대신 해주고 신원을 헤더로 넘겨주는 기능.
CORS	어느 웹사이트(프론트)에서 이 서버를 호출해도 되는지 허용목록.
SSE / 스트리밍	결과를 한 번에 주지 않고 글자 단위로 실시간 전송(타이핑 효과).
CI/CD	코드를 올리면 자동으로 빌드·배포되는 파이프라인.
TTL	데이터의 유효기간. 지나면 자동 삭제(캐시 24시간).

BYBAEK 시스템 정의서 & 배포 명세서 · v1.0 · 기준일 2026-07-14 · 대상 **bybaek-backend**
(feature/onedrive-token-fix)

본 문서는 실제 소스코드를 직접 분석해 작성되었습니다. 코드 변경 시 해당 장을 함께 갱신하세요. 시크릿은 포함되어 있지 않습니다.

